P4X 双端遥测系统 • 答辩技术手册（详解版）

标注图例：★ 主办方高频 / 必须背下来　▲ 要能讲清原理　○ 知道即可

使用方法：第 0 章背熟；第 1–5 章按图讲；第 6–7 章是"挡问区"；第 8 章防"怎么复现"；第 10 章是**按评委视角整理的问题清单**（比第 9 章的按技术分类更贴近现场）。

---

0. 开场白与数字底牌 ★

0.1 30 秒开场（原文照背）

> "我们做的是**探空火箭双端遥测系统**。箭上是一块自研的单芯片 ESP32-S3 整机板，只做采集、组帧、存储、上链；地面是一块跑 **openvela（NuttX）** 的 ESP32-P4X 终端，1024×600 触摸屏、中文界面，能看到姿态/高度/卫星，也能把点火、灭火、结束这三类关键指令确认下发。 空口只有 960 B/s，我们用短键 JSON 把占用压到 455 B/s；高度抖动从 3.18 m 压到 1.61 m；连续 90 秒零异常字节。"

0.2 必须记住的 8 个数字 ★

数字	含义	一句话解释
960 B/s	A28 LoRa 9600 8N1 的理论带宽	9600 bps ÷ 10 bit/字节 ≈ 960 B/s
455 B/s	实际占用	quick 26B×4Hz + full 175B×2Hz
3.18 → 1.61 m	高度抖动（卡尔曼前后）	抖动=静止时高度的峰峰值波动
13.95 → 2.12 m	高度最大跳变	相邻两帧之间的最大差值
1 s	半双工纯接收窗口	发射时听不见，必须留时间收命令
14 × 120 ms	命令重发序列	跨约 1.7 s，覆盖瞬时遮挡

数字	含义	一句话解释
30 fps	局部刷新帧率	只重绘脏矩形，不整屏重绘
0	90 s 采样异常字节数	稳定性硬指标

---

1. 系统全景 ★

1.1 一句话人话版

> "箭上负责**采得准、存得住、发得出**；地面负责**看得懂、点得对**。中间用最省的带宽说话。"

1.2 数据流（按这个顺序讲，不要跳）



1.3 职责边界为什么要这么切 ★

- 箭上做**确定性**的事：采样、组帧、落盘、发送 —— 不需要人参与，越简单越不容易崩；
- 地面做**需要人判断**的事：渲染、研判、下发 —— 允许慢一点，但要看得清。
- **好处**：任一端挂掉，另一端仍能独立工作（箭上照样记录，地面照样回放）。

---

2. 硬件

2.1 形态：两块同径圆盘 ★

盘	装什么	为什么放这一盘
主控盘	ESP32-S3-WROOM-1-N16R8 + A28 LoRa + USB-C + 电源/充电 + TF 卡座	大电流、发热、天线都在这里，散热与射频好处理
传感盘（薄）	MPU6050 + 气压计 + GPS 接口 + OLED/触摸预留	贴质心、走线最短、板薄可层叠

分舱的三个理由（评委常追问"为什么不一块板"）：

1. 细箭体里单板面积塞不下；
2. 换传感器只改传感盘，主控盘不动（迭代快）；
3. 坏一块换一块，不整机报废。

必须主动说的代价：板对板连接器是新增失效点、层叠散热差。

2.2 引脚分配（★ 必背 6 行，其余能查）

功能	引脚	约束
蜂鸣器 F+	**IO17**	阶段音效，串限流/三极管
LoRa A28	**TX IO14 / RX IO21**	9600 8N1，MD0/MD1 接地
六轴 IMU	**SCL IO40 / SDA IO39**	与气压共用一组 I²C
GNSS（GP21 多模）	**RX IO10 / TX IO4**	115200 8N1，模块 TXD→IO10，$GN 多系统语句
TF 卡	**CS7 SCK5 MOSI6 MISO11**	独立 SoftSPI，MISO 上拉

功能	引脚	约束
OLED	**SCL IO48 / SDA IO42**	本地状态显示
阶段灯 / 电源灯	IO38 / IO2	WS2812 / 心跳
原生 USB	D− IO19 / D+ IO20	USBLC6 ESD，禁止复用
调试串口	IO43 / IO44	CH340K
保留空置	**IO0 / IO3 / IO45 / IO46**	启动脚，外部不得上拉

2.3 三类"隐形雷区" ★★（最容易被追问的一节）

雷区	为什么危险	我们怎么处理
启动配置脚 IO0/IO3/IO45/IO46	上电瞬间的电平决定"从哪里启动"和 flash 供电电压	四脚全部空置，外部不接任何上拉
八线 PSRAM 占 IO33–IO37	N16R8 的 PSRAM 在封装内部接这几根线，对外不可用	LoRa 从 IO35/IO36 迁到 IO14/IO21
原生 USB 占 IO19/IO20	接了 ESD 保护就不能再当普通 IO	TF 的 MISO 从 IO20 迁到 IO11

术语人话版 ▲（不懂这些词会被问穿）

- **Strapping pin（启动配置脚）**：芯片上电那一瞬间会去"量"几个脚的高低电平，用来决定"从 flash 启动还是进下载模式""flash 用 1.8 V 还是 3.3 V"。这些脚一旦被外部电阻拉高，芯片就会理解错，表现为"烧不进去 / 起不来"。
- **八线 PSRAM**：外挂内存用 8 根数据线，速度快，但把 IO33–IO37 全部占死；R2 版本（四线）才只占 4 根。我们用的是 N16R8 = 16 MB flash + 8 MB 八线 PSRAM。

- **eFuse**：芯片内部一次性的"保险丝位"，模组出厂会把 flash 电压烧死，所以 IO45 在模组上不会真改电压 —— 但这是"运气"，不是设计依据，所以我们仍然空置。

2.4 其它硬件决策 ▲

项	我们的做法	为什么
电源	TPS63020 升降压 + 0.5 A PTC	覆盖 3.0–4.2 V 电池，扛住 LoRa 发射的 200 mA 脉冲
充电	TP4056 + USB-C	5 V 输入直接充单节锂电
CC 电阻	CC1/CC2 各 **5.1 kΩ 下拉**	Type-C 做设备端必须下拉，否则 C-to-C 线不给电（自制板 90% 踩这个坑）
USB 防护	USBLC6-2SC6 贴连接器	D+/D− 走 90 Ω 差分，防静电
WS2812	5 V 供电 + 3.3 V 数据	注意 WS2812B 的 VIH≈0.7×VDD=3.5 V，3.3 V 边缘擦线，可改 3V3 供电或加电平转换

---

3. 空中协议 ★★

3.1 两种帧（背下来）

帧	周期	大小	内容
quick	4 Hz	26 B	p / r / y 姿态

帧	周期	大小	内容
full	2 Hz	175 B	姿态 + 经纬度(6 位小数) + 加速度 + 融合高度 + GPS/卫星 + 温度 + 气压 + 速度 + 时间 + 链路 + 阶段 ph

合计 ≈ 455 B/s，链路 960 B/s，余量近一半。

3.2 短键表 ★

`p` 俯仰 `r` 横滚 `y` 偏航 `a` 融合高度 `P` 气压（**大写，小写 p 是俯仰**）`t` 温度 `la/lo` 经纬度 `gps/s` 定位与卫星 `ph` 阶段 0–4 `L` 链路 1/0/−1 `T`GPS 时间

3.3 为什么 JSON 不用二进制 ▲

- **好处**：可读、可现场用串口看包、字段可随意增减（比赛阶段排错第一）；
- **代价**：比二进制多 30–40% 体积；
- **怎么补**：字段名压到 1–2 个字符 + 分帧率（姿态高频、全量低频）。
- 如果评委问"能不能更省"→ 可以，但会失去现场可读性，比赛阶段不划算；上量产后可切二进制 + 协议版本号。

3.4 半双工为什么必须留 1 s 窗口 ★★（高频）

- LoRa 是**半双工**：发的时候收不到；
- 960 B/s × 1 s = 960 B，而 UART 只有 128 B 硬件 FIFO、MicroPython 侧 256 B rxbuf；
- **窗口期进来的命令会整条丢失**（实测 400 ms 窗口下命令成功率明显下降，1 s 才稳）；
- 我们还在窗口期内**禁止写 SD 卡**（一次写卡阻塞 300 ms，等于丢掉半条命令）。

3.5 命令可靠投递（三段式）★★

```
地面：按住 2 s 确认 → 14 包 × 120 ms 连发（跨 1.7 s）
[redacted]
箭上：按"阶段是否变化"去重 → 只执行一次（避免重复点火音效）
地面：等遥测里的 ph == 目标阶段 → 显示"发送完成"
[redacted]
      15 s 无回报 → 显示 FAIL，且屏幕不自行跳阶段
```

为什么去重按"阶段"而不是按"时间"：按时间会把"重新下发同一阶段"误吞；按阶段则是"状态变了才执行"，与物理状态一致。

---

4. 箭载端固件（MicroPython）

4.1 主循环顺序 ★

收命令（最高优先）→ 蜂鸣器节拍 → 全双工兜底 → 遥测发送 → SD 记录 → 屏幕。

所有耗时动作都拆成非阻塞：蜂鸣器 `play()` 只排队不 `sleep`，否则一段旋律能把主循环卡 700 ms。

4.2 SD 记录参数表 ★★（每个数字都会被问）

参数	值	为什么
记录频率	200 ms（5 Hz）	单次 `open→write→close` 实测 120–300 ms（FAT 开销），10 Hz 会拖死主循环
攒批	20 行/次	把写卡开销摊薄到约 15%
挂载时钟	200k → 100k → 400k 自适应	400 kHz 下 CMD0 能应答但读 CSD 失败，且**失败会把卡打进不应答状态**
写卡让位	接收窗口内不写	一次写卡 300 ms = 丢半条命令
失败处理	5 次错误停录，但保留重挂	每 20 s 自动重挂，卡恢复自动续录
文件名	`telemetry_<N>.csv` 取空闲号	绝不覆盖上一架次
阶段联动	**收到"预备"新建文件开始记；收到"结束"落盘停止**	一个架次一个文件

参数	值	为什么
无命令兜底	高度比基线抬升 >3 m 自动切新文件	地面端不发命令时也能分段；只切不停，绝不截断

术语人话版：

- **FAT 开销**：往卡上写一行不是"写一行字"，要先读 FAT 表、找空簇、改目录项、再写数据，所以有固定几百毫秒；
- **攒批**：把 20 行攒在内存里，一次写下去，把"固定开销"摊薄 20 倍；
- **写卡让位**：写卡时 CPU 全在等 SPI，收不到无线命令，所以要错开。

4.3 高度融合（卡尔曼 + 海拔）▲★

现在这一版（实机在跑）：

- 预测：`h += Δ(气压海拔)` —— 气压负责**快变**，短时间很准；
- 观测：GPS 的 **MSL 海拔**，`R = 400`（很大）→ 只用来消慢漂，GPS 噪声进不了输出；
- 输出：**相对海平面的绝对海拔**（不是相对起飞点的相对高度）；
- 丢星时：保持当前卡尔曼状态，不重取基准、不跳变。

为什么 GPS 定高不可靠（评委常问）：

量	实测	说明
参与定位星数	10–12	看着不少
HDOP	1.6–2.7	水平精度因子
VDOP	≈1.5–2 × HDOP	垂直天然更差
垂直误差	**±8–15 m**	所以 GPS 给不了米级高度

结论：**气压定高、GPS 定位置**，这是行业做法，不是我们做不到。

早期版本（已废弃，作为"踩坑"可讲）：

- 曾直接写 `fused_alt = 气压相对高度 + GPS 绝对海拔`，静止时显示 −17 m 且乱飘；

- 也曾把气压基准取成"开机时的当地气压"，于是输出变成相对起飞点的高度，而不是海拔。

术语人话版：

- 卡尔曼滤波 = "预测 + 修正"：先按上一时刻猜一个值（预测），再拿新测量去纠正（修正），两者加权；
- **Q（过程噪声）**大 → 更相信测量、跟得快、但抖；**R（观测噪声）**大 → 更相信预测、平滑、但迟钝；
- 我们把 GPS 的 R 取到 400，就是因为它的垂直误差实在太大；
- 地面站另有一套 1-D 卡尔曼（Q=9 / R=2.25 / 150 m 野值门限）用于曲线拟合，抖动 3.18 → 1.61 m。

4.4 GNSS（GP21 多模多频）▲

模块：GP21 多模多频接收机，同时收 **GPS + 北斗 + GLONASS + Galileo**，语句前缀是 `$GN`（多系统融合解），不是单系统的 `$GP`。

接线与波特率（踩过的坑）：

项	值	说明
模块 TXD	→ **IO10**（ESP32 的 RX）	一开始接反成 IO10=TX，收不到任何字节
模块 RXD	← **IO4**（ESP32 的 TX）	只用来发配置，可不接
波特率	**115200 8N1**	不是常见的 9600，9600 下只会看到乱码
电平	3.3 V	与 S3 直连，无需电平转换

只解析 GGA + RMC + GSV：GGA 给定位状态、**参与定位星数**与经纬度，RMC 给时间与速度，GSV 给**视野内**每颗星的 CN0。经纬度 6 位小数（≈0.11 m），发送与解析都用 double（float 只有约 7 位有效数字，会掉精度）。

两个数不能混（现场最容易被问）：

指标	来源	含义
参与定位星数 `sat`	GGA 字段 7	真正进入解算的星

指标	来源	含义
视野内星数 `satv`	GSV 各星座相加	只是"听得到"
定位质量码 `q`	GGA 字段 6	0 无效 / 1 单点 / 2 差分 / 4 RTK 固定 / 5 RTK 浮点 / **6 估算**
HDOP	GGA 字段 8	<2 好，>5 差；垂直误差还要再乘 1.5–2

现场实测（室内）：`q=1、sat=10~12、satv=6~9、HDOP=1.6~2.7`、最强 CN0 35 / 平均 26–33 —— 这种条件下水平定位可用、**垂直误差仍有 ±8–15 m**，所以高度以气压为主。

冷启动与验收：室内必然 0 星（`fix=0 sat=0`），这是正常的；到室外空旷处冷启动 **30–120 秒** 出定位。验收看四件事：`$GNRMC` 的 A/V 状态位、GGA 的卫星数、质量码是否变 1、HDOP 是否降到 2 以下。

一个真实的干扰案例：天线正前方 1 cm 处有一颗 **M3 金属螺杆**，属于天线近场导体，会拉偏谐振、压低增益。处理顺序：① 换尼龙螺丝（最省事、彻底）；② 金属必须保留时用尼龙套管隔开并缩短螺杆，让金属顶端低于天线上表面；③ 把天线挪到距金属 ≥20 mm。**不要**在天线与螺杆之间贴铜箔/铝箔（等于挡天空），也不要贴吸波片（近场直接吃增益）。

---

5. 地面站（openvela / NuttX）

5.1 任务与优先级 ★（ps 实测，可现场演示）

```
PID 3   PRI 100  p4x_gs      主任务（解析 + UI）
PID 2   PRI 200  tchsamp     触摸采样
PID 5   PRI 110  lora_rx     串口收包
PID 6   PRI 112  panel       面板看门狗
```

为什么这么排：panel 最高（掉帧就蓝屏，必须优先救）→ lora_rx 次之（收包不能被 UI 卡）→ p4x_gs 做重活（数字越小优先级越低）。

5.2 驱动攻坚四步 ★（按真实调试顺序讲，很有说服力）

1. **UART1 打通**：四处根因 —— 波特率时钟分频（直写 CLKDIV）、信号矩阵 `SIG_IN_SEL` 选路、引脚复用使能、`rxfifo_cnt` 判据；

2. **面板稳定**：冷启动只有背光 → 根因是 **syslog 写 USB 控制台在无读者时阻塞启动**，用 **RAMLOG**（日志进内存）解决；运行中蓝屏 → panel 任务每秒检查"帧有没有推上去"，1 s 无新帧就重放面板上电时序；
3. **触摸可用**：拖拽阈值 12 → 30 px、滑动防抖 40 ms–2.5 s（正常按压的轻微移动会被误判成滑动手势）；
4. **中文 UI**：自研点阵字库 + 离线渲染工具链 —— 缺字会被**静默跳过**，所以必须有工具能查缺字。

术语人话版：

- **CLKDIV**：串口波特率 = 时钟 ÷ 分频，NuttX 里要直接写分频寄存器，否则波特率差一点就收乱码；
- **信号矩阵**：芯片内部把"哪个引脚接到哪个外设"，配错了引脚就是没信号；
- **RAMLOG**：系统日志默认往 USB 控制台打，没人读时会卡住启动 —— 改成写进内存环形缓冲就没事了；
- **看门狗**：定时检查"心跳还在不在"，不在了就把外设重新初始化。

5.2b P4X 适配：寄存器级细节（评委最爱追的部分）★

这块板子在 openvela 里**没有现成 BSP**，下面四处都必须自己查到寄存器 / 驱动层：

① UART1 收不到字节 —— 四个独立原因叠加

层次	具体问题	怎么定位
时钟	波特率由 `CLKDIV` 直接分频，只填"波特率数值"不够	9600 收出来是乱码，量位宽对不上
选路	引脚到外设要过信号矩阵 `SIG_IN_SEL`	引脚有电平变化，FIFO 里没数据
复用	IO MUX 的 FUNC 没使能，脚还是 GPIO	回读 MUX 功能号仍是 0
判据	"有没有数据"不能只读 `rxfifo_cnt`	计数非 0 但读出全 0

四处必须**一起满足**：只修其中一处，现象完全一样（没有数据），这也是它最难查的原因。

② 面板冷启动只亮背光

- 现象：上电背光白屏、NSH 都不出来；
- 根因：**syslog 往 USB 控制台写，没有读者时会阻塞**，把启动卡死；
- 解法：**RAMLOG**（日志进内存环形缓冲）+ `panel` 任务看门狗重放面板上电时序；
- 附带坑：上位机打开串口时若**拉高 DTR/RTS**，会把 SoC 压在复位状态，表现同样是"屏不亮"。

③ 触摸"点不准、滑不动"

- 拖拽阈值 **12 → 30 px**：手指按下时的微小抖动会被判成滑动，导致点击丢事件；
- 滑动防抖 **40 ms – 2.5 s**：太短当误触，太长像卡住；
- 触摸采样独立任务（优先级 200，全场最高），否则 UI 重绘期间点击延迟能到几百毫秒。

④ 中文显示

- 面板无中文字库 → 自研 **2 值点阵字库** + 离线渲染工具（TTF → C 数组）；
- 缺字是**静默跳过**（不报错、只是不显示），所以必须有工具把用到的字扫一遍。

⑤ 任务与构建

任务	优先级	职责
`tchsamp`	200	触摸采样（最高，保证手感）
`panel`	112	面板看门狗（无新帧就重放上电时序）
`lora_rx`	110	串口收包（高于渲染，防字节丢失）
`p4x_gs`	100	解析 + 渲染主任务

构建链路：`Kconfig` 声明配置 → 板级 `defconfig` 打开 → `CMakeLists` 注册应用 → `cmake --build` 出 `nuttx.bin` → `esptool` 烧录（**BOOT 要按住**，脚本自动重试）。

5.3 为什么能 30 fps ★

整屏 1024×600×2 B ≈ **1.2 MB**，全屏重绘来不及。改成**脏矩形局部刷新**：只重画变化的矩形（实测约 190×102）并 `FBIO_UPDATE` 推送，达到 30 fps。

术语人话版：脏矩形 = 只把"这一块变了"的区域重新画并推给屏幕，而不是每次重画整张图。

5.4 交互一致性 ▲

阶段配色与箭载端 RGB 灯一一对应（绿/橙/红/紫/白）；"预备"阶段兼作定位灯：有定位绿、无定位粉 —— 让"屏幕上的颜色"和"箭上的灯"永远是同一个含义。

5.5 配套网页的内置 AI 助手 ▲

它是什么：网页右上角「AI 助手」，一个能读当前遥测、能连续追问的对话面板。用来在联调和答辩现场快速定位问题，不用翻手册。

数据怎么走：



为什么中间要加一层代理：API Key 只存在于后端进程（环境变量或 `~/.dsh/.credentials.yaml`），前端页面拿不到；同时把跨端口的请求收成同源，避免浏览器 Private Network Access 拦截。

它每次拿到什么：系统提示词（项目事实：单芯片、9600 bps、8 Hz、互补滤波、已知短板）+ 页面实时快照（高度/速度/卫星/LoRa/姿态/阶段/控制台尾部日志）+ 你这次的问题 + 最近 10 轮对话。所以数据的数值变了，结论就跟着变；追问也接得上。

边界（要主动说）：依赖网络，断网不可用；它不直接读串口，看到的是页面喂给它的数据；它不知道训练集以外的现场情况，遇到数据不足会反过来说明"需要看哪几个量"。

答辩里的两句话：

> "网页不只是显示面板：AI 助手读的是同一套遥测，能直接把'现在高度抖、卫星 0'翻译成'先查哪三件事'。" "Key 放在后端代理，前端只拿结论 —— 这也是这个功能在工程上该有的隔离。"

---

6. 十个关键取舍 ★★（每个都要能说出代价）

决策	选了什么	代价 / 边界
箭上主控	ESP32-S3（便宜、脚位足、MicroPython 生态）	无 MIPI、算力跑不动大屏
地面主控	ESP32-P4X（有 DSI 和大屏算力）	无 Wi-Fi 需外挂，资料少坑多
无线	A28 2.4G LoRa（3–5.5 V 内置 LDO、200 mA 脉冲余量）	绕射差、依赖视距、半双工
帧格式	短键 JSON	比二进制多 30–40% 体积
SD 总线	SoftSPI + 200 kHz	慢，但可避开总线冲突、时序可控
记录频率	5 Hz	不是 10 Hz，因为写卡固定开销 300 ms
半双工	1 s 纯接收窗口	牺牲约 1/3 吞吐换命令可达性
高度算法	1-D 卡尔曼 + 150 m 门限	极端机动（>10 g）模型失配
界面刷新	脏矩形局部刷新	整页换图仍需全刷
硬件形态	两块同径圆盘	连接器是新失效点、散热要设计

---

7. 已知局限（主动交底，别等评委挖）★

1. **面板物理层不稳**：DSI 排线碰一下就蓝屏，软件只能"掉了再拉起来"，根治要固定排线 / 降 DSI 时钟；
2. **界面偏"工程风"**：暗底、信息密集，缺品牌感 —— 已有整改计划；
3. **SD 卡座接触不稳**：老卡写一会儿就挂（读正常、写被拒），已做自适应时钟 + 自愈重挂；
4. **PSRAM 不可用**：老板 PSRAM 初始化失败（bail out），只能靠片上内存 + 分帧设计；
5. **2.4G 视距限制**：>2 km 明显衰减，远距要换 433M 或图传；
6. **地面端当前不下发 `B:` 阶段命令**：靠箭上"高度自动分段"兜底，这是下一步工作。

---

8. 配置与构建框架（防"怎么复现"）★

8.1 openvela 侧

```
# 必须重新 configure：手改 .config 不会触发 config.h 重生成
cmake -B cmake_out/<board> -S nuttx -GNinja \
  -DBOARD_CONFIG=<board>/configs/nsh \
  -DEXTRA_FLAGS='-Wno-cpp -Wno-deprecated-declarations'

cmake --build cmake_out/<board>
```

应用接入：`apps/examples/<app>/{Kconfig, Makefile, Make.defs, CMakeLists.txt}`，板级 defconfig 里加 `CONFIG_EXAMPLES_<APP>=y`。

踩过的坑：改配置前必须备份板级 `defconfig`（曾因写入异常把 defconfig 截断成 2 行，整个构建损坏）；烧录前要停掉所有占用串口的程序。

8.2 箭载端

```
pip install mpremote pyserial
python -m mpremote connect COM7 cp src/central_tx_s3.py :central_tx.py
python -m mpremote connect COM7 soft-reset
```

`main.py` 保持 `import central_tx; central_tx.main()` 即**开机自启，不依赖 WiFi/PC**。

8.3 一键工具（`tools/`）

脚本	作用
`p4x.py` / 启动 `P4X.bat`	地面站启动 / 状态 / 重启 / 网页
`sd_status.py` / `sd_space.py` / `sd_probe.py`	看卡、看空间、CMD0 与多时钟挂载诊断
`_test_fsm.py` / `_test_pair.py`	记录状态机自测（预备新建 / 结束停止）
`fwid.py`	查板上跑的是哪版固件（ps 任务表指纹）

---

9. 高频技术 Q&A（按技术分类）

架构

Q：为什么不用现成地面站软件?

A：专业遥测站几万元起，还要 PC + 许可；我们要"掌上、开机即用、能确认执行"。代价是驱动和 UI 全部自己扛。

Q：为什么两端不用同一颗芯片?

A：需求相反。箭上要省电、可靠、脚位足；地面要大屏、GPU、触摸。S3 做箭上、P4X 做地面，各自最优。

Q：单芯片板比模块拼装强在哪?

A：引脚可规划（避开启动脚/PSRAM/USB）、线束少约 80%、装配一致、电源与存储一次定型。

硬件

Q：IO45/IO46 为什么不能用?

A：启动配置脚。IO45 决定 flash 供电电压，上电必须低；IO46 参与下载模式判定。

Q：PSRAM 为什么用不了/为什么还能用 IO35?

A：八线 PSRAM 封装内占 IO33–IO37；我们这颗 PSRAM 初始化失败后放弃使用，所以 IO35 恰好能当普通 IO —— 那是运气，不是设计，所以 LoRa 已迁走。

Q：USB-C 为什么要 CC 电阻?

A：Type-C 设备端必须 CC1/CC2 各 5.1 kΩ 下拉，否则 C-to-C 线不给电。

协议 / 链路

Q：960 B/s 够用吗?

A：够，占用 455 B/s，余量近半；再高就得牺牲帧率或改二进制。

Q：丢包怎么办?

A：三层 —— ① 命令 14 包重发；② 箭上按阶段去重；③ 数据本身走 SD 本地记录兜底。

Q：为什么必须等 ph 回报?

A：半双工没有 ACK 就没把握；以"遥测 ph == 目标阶段"为唯一成功判据，超时 15 s 判 FAIL，屏幕不自行跳阶段。

固件

Q：为什么用 MicroPython?

A：迭代速度。性能瓶颈都在硬件外设（SPI/UART），不在解释器。

Q：5 Hz 不是 10 Hz?

A：写卡固定开销 120–300 ms（FAT），10 Hz 会把主循环吃掉。

Q：卡坏了程序会死吗?

A：不会。停录 + 每 20 s 自动重挂，主循环与遥测不受影响。

地面站

Q：openvela 上你们改了什么?

A：板级 bringup、UART1 驱动、MIPI-DSI 面板与看门狗、GT9xx 触摸、点阵字库、应用层多任务与局部刷新，全部落在板级与 `apps/examples` 内，可复现构建。

Q：30 fps 怎么做到?

A：脏矩形局部刷新 + `FBIO_UPDATE` 推送，而不是整屏 1.2 MB 重绘。

Q：中文怎么显示?

A：自研点阵字库，按字取模；缺字会静默跳过，所以配了工具查缺字。

---

10. 按评委视角整理的问题清单 ★★★（现场最可能问）

10.1 技术深度 / 会不会

1. 你们的链路上限是多少？现在用了多少？怎么算出来的？★
2. 半双工窗口为什么是 1 秒，不是 0.4 秒？★
3. 卡尔曼的 Q/R 怎么定的？换个传感器还成立吗？
4. 为什么用软件 SPI 驱动 SD 卡？速度损失多少？
5. 高度为什么不做海拔换算？★
6. 两个 MCU 之间为什么不做时间同步？（答：都打 GPS 时间戳，无 GPS 时用相对序号）
7. 断链 5 分钟再恢复，数据会不会乱序/重复？（答：ph 与时间戳双判据）
8. 为什么姿态用互补滤波而不上完整的姿态解算？
9. 你们怎么保证写卡不丢数据？（攒批 + 让位 + 自愈重挂 + 只切不停）
10. 现在的帧率再翻倍，会先崩在哪？（答：写卡与半双工窗口）

10.2 完成度 / 稳定性

1. 连续跑多久算通过？★（答：90 s 零异常字节；另有长时间挂机观察）
2. 屏幕为什么会黑？你们怎么定位的？★（RAMLOG + 看门狗 + 重试）
3. 现场演示要是翻车怎么办？（答：拔插 USB 15 s 恢复、SD 卡数据兜底）
4. 有没有做异常注入测试？（答：拔天线、拔卡、断电、灌噪声）
5. 烧录要按 BOOT 吗？（答：是，这是当前固件的限制，已列入整改）

10.3 创新性 / 差异化 ★

1. 你们和"买个 LoRa 模块 + 串口助手"的本质区别是什么？★

→ 区别在**闭环**：阶段命令可确认执行、数据可复盘、断链不丢、界面能现场判断。

1. 短键 JSON 算创新吗？（答：不算算法创新，是**工程约束下的最优解**，且可量化到 455/960）
2. 高度滤波算创新吗？（答：数值不算新，但"因为抖动太大导致阶段判断失败"这个工程问题是真实的，方案是实机调出来的）

3. 你们的门槛在哪？（答：驱动层 —— UART1 打通、DSI 面板稳定、中文渲染，这些是最难复现的部分）

10.4 工程规范

1. 代码怎么管理？（答：git，单提交、分支 `feature/p4x-telemetry`、`.gitignore` 排除固件/日志/第三方 PDF）
2. 文档齐不齐？（答：README + 接线表 + 交接记录 + 答辩手册 + 演示思路）
3. 别人能复现吗？★（答：能，构建命令、上传命令、一键脚本都在）
4. 有没有测试？（答：状态机自测脚本、SD 诊断脚本、90 s 稳定性验收脚本）

10.5 安全 / 合规

1. 点火指令怎么防误触？★（答：按住 2 s 红框填满才发、5 s 冷却、15 s 超时 FAIL、屏幕不自行跳阶段）
2. 2.4G 发射功率合规吗？（答：按当地法规预设功率/信道，量产需认证）
3. 断电/炸机后数据还在吗？（答：SD 卡本地记录，攒批最多丢 4 s）

10.6 商业 / 落地

1. 成本多少？（答：百元级 BOM，具体按批次）
2. 卖给谁？（答：教学/科研/爱好者；中期做探空火箭标准遥测模块）
3. 为什么不用手机 App 做地面端？（答：现场环境、阳光下可读、单手操作、断网可用）
4. 你们最大的风险是什么？★（答：面板与卡座的物理可靠性，已列入整改）

---

11. 名词与单位速查

名词	一句话
strapping pin	上电时决定启动模式的引脚
eFuse	芯片内一次性配置位（如 flash 电压）

名词	一句话
八线 PSRAM	用 8 根数据线的外挂内存，占 IO33–IO37
半双工	收发不能同时，需要预留接收窗口
脏矩形刷新	只重绘变化区域
RAMLOG	系统日志写内存，避免阻塞启动
Q / R	卡尔曼的过程噪声 / 观测噪声
FIFO	串口硬件缓冲（本项目 128 B）
CLKDIV	串口时钟分频，决定波特率
ph	飞行阶段索引 0–4

---

12. 演示脚本与翻车预案

正常演示（90 秒）

1. 上电 → 地面端自动探测 A28 → 遥测秒级上屏；
2. 按住"下一阶段"2 s → 红框填满 → 连发 → 箭上 RGB 变红、屏幕阶段同步；
3. 拔天线 15 s → 界面转 FAIL → 恢复后自动重连。

翻车预案

现象	现场话术	处理
屏幕黑	"面板冷启动只有背光，是我们已知的物理层问题"	双击 `启动 P4X.bat`，或拔插 USB 等 15 s
没遥测	"链路断了，但箭上照样在记录"	讲断链设计 + 拿 SD 卡数据

现象	现场话术	处理
卡不认	"卡座接触问题，我们做了自愈重挂"	换卡或重插
设备卡顿	—	直接讲数字与架构，别等设备

---

13. 三句话收尾

1. **架构**：箭上做确定性、地面做可视化，中间用 455/960 B/s 讲清楚。
2. **工程**：每个数字都能复跑，每个坑都写进了文档。
3. **产品**：把"专业站才有的能力"压到千元级，让更多学校做得起一次有遥测的飞行。

---

14. 研发背景与开放题（评委最常问的"为什么"）★

> 前面 13 章是"怎么做的"，这一章是"**为什么做、值不值得做**"。 技术问题答不好只是扣分，这一章答不好会让人怀疑"这事有没有意义"。

14.1 60 秒版"为什么做这个"（可直接背）

> "现在商业航天正在从'国家工程'变成'高频次、低成本'的产业——可回收火箭把单位入轨成本压到了过去的零头，发射次数一年比一年多。 但**热的是产业，冷的是科普**：青少年能看到新闻，却很少有机会真正'动手做一次航天实验'。 模型火箭是最合适的入口：便宜、安全、能飞。可它有个共同痛点——**飞完只有'哇，上去了'，没有数据**。 因为没有数据，就没法复盘；没有复盘，就不是工程训练，只是表演。 而**像样的遥测设备，动辄上万元**，学校买不起。 所以我们做的不是'又一个玩具火箭'，而是**一套千元级、掌上、而且是'指令可确认'的遥测系统**—— 让'飞一次'变成'做一次实验'。 "

14.2 八个"为什么"（每个都能被追问，答一句就够）

问题	答法
为什么是模型火箭，不是无人机?	无人机门槛在"飞手"，模型火箭门槛在"数据"。我们的能力正好补后者的空白；而且火箭的飞行窗口只有几十秒，对数据可靠性要求更高，技术含量更实。
为什么自己做板，不买现成模块?	第一版就是模块拼装，6 块板 + 十几根线，引脚互相抢占、现场一碰就掉。做板是为了**让"能跑"变成"能交付"**。
为什么箭上用 MicroPython?	迭代速度。性能瓶颈都在硬件外设（SPI/UART/定时），不在解释器；而赛场上"改一行就能试"比"省 3 ms"重要。
为什么 2.4G 不用 433M?	我们要的是**小天线 + 高一点速率**：455 B/s 的遥测在 433M 低速率下会挤爆；距离用视距来换，够用。
为什么做中文界面?	目标用户是中学老师和学生。**看得懂比看得炫重要**，英文菜单会直接劝退一半人。
为什么反复强调"可确认"?	火箭只有一次机会。命令发出去不等于执行了 —— 我们用 ph 回报闭环，把"我以为"变成"我看到"。
为什么开源?	这类工具的价值在"能被复现和改"。开源让别的学校能直接拿去用，也让我们的坑变成别人的路。
为什么现在不做云端?	现场常常没网络。**离线可用**是这个场景的第一需求，云端是锦上添花。

14.3 应用型开放题（PPT 里放不下，现场被问就用这套答）

Q：目标用户到底是谁？先打哪个场景?

A：第一优先是**中学/高校的航天社团与竞赛队**（有老师、有场地、有明确经费），第二是**科研院所的探空试验**（要低成本快速验证），第三才是爱好者。先打教学，因为教学场景对"便宜 + 好懂"最敏感，也最容易形成口碑。

Q：怎么进校园？不只是卖硬件吧?

A：三件套——① 套件（硬件 + 组装说明）；② 课程包（8 节课：原理 / 组装 / 遥测 / 数据处理 / 复盘）；③ 教师培训 + 安全规范（燃料用量、发射场地、撤离距离）。**硬件只是入口，课程才是留存。**

Q：和商业遥测产品比，你的差异？

A：商业产品卖"性能与资质"（几十公里、抗干扰、认证齐全），我们卖"**便宜、能改、能看懂**"。我们不抢他们的客户，我们服务那些**根本不会买商业设备**的人。

Q：百元级成本是怎么做到的？

A：三件事——① 单芯片集成（省掉 5 个模块的连接器与外壳）；② 用成熟国产器件（A28、ME6211、CH340K）；③ 软件补硬件（短键协议省带宽、卡尔曼省传感器、局部刷新省显存）。每一项都能在 PPT 里对应到一页。

Q：怎么持续？总不能一直开源白送。

A：三层：**开源固件（建立信任）→ 套件销售（覆盖成本）→ 课程与培训（产生利润）**。开源不是商业模式，是获客方式；真正卖的是"能开课的完整方案"。

Q：安全和合规怎么办？

A：分三块——① **燃料与发射**：固体发动机按当地规定采购与存放，发射场地要报备，人员按撤离半径管理；② **无线**：2.4G 功率与信道按当地法规预设，量产需做认证；③ **教学安全**：课程里固定"三查三不"（查电池、查结构、查场地；不超量、不超角、不超风速）。

Q：如果给你 6 个月和 10 万，先做什么？

A：三个月做**可靠性**（面板物理层、卡座、自动下载），两个月做**课程包**，一个月做 **10 套小批量**给 3 所学校试用。**先证明"老师能独立开课"，再谈规模。**

Q：团队怎么分工？

A：（按实际填）硬件 1 人（原理图/PCB/焊接）、嵌入式 1 人（箭载固件）、系统与 UI 1 人（openvela 地面站）、测试与文档 1 人（也是我）。四个人、三个月的产出就是这套东西。

Q：最大的风险是什么？

A：不是技术，是**"做出来没人用"**。所以我们现在最看重的是"老师能不能不看说明书就开一节课"——这也是我们把界面从"工程师风"往"产品风"改的原因。

Q：怎么证明科普效果？

A：三个可量化指标——① 一节课内学生能独立完成组装与一次记录；② 课后能读懂自己那张高度曲线；③ 有学生愿意在下一节课改一个参数再飞一次。**第三点最重要，那是工程思维的开始。**

14.4 一句话收尾（背下来）

> "我们做的不是一颗火箭，是**让一次飞行产生数据、让数据变成一节课。**"